

Guía Completa de Git y GitHub

1. Fundamentos de Git

Git es un Sistema de control de versiones distribuido (DVCS). A diferencia de otros sistemas, Git no solo descarga la última copia de los archivos, sino que replica el repositorio completo en la máquina local del usuario.

Características Principales

-Snapshots (Instantáneas): *Git maneja la información como un conjunto de copias instantáneas de un sistema de archivos miniatura. Si un archivo no se ha modificado, Git guarda un enlace al archivo anterior idéntico en lugar de duplicarlo.*

-Operaciones Locales: *La mayoría de las acciones no requieren conexión a red, lo que permite una velocidad de trabajo casi inmediata.*

-Integridad: *Utiliza el algoritmo SHA-1 (una cadena de 40 caracteres) para crear una suma de comprobación de cada objeto, garantizando que nada se pierda o se corrompa sin que Git lo detecte.*

-Los Tres Estados de Git

Para gestionar el flujo de trabajo, los archivos atraviesan tres secciones específicas:

Modificado (Modified): El archivo ha cambiado, pero aún no se ha guardado en la base de datos.

Preparado (Staged): El archivo se ha marcado en el "área de preparación" (index) para incluirse en la próxima confirmación.

Confirmado (Committed): Los datos están almacenados de forma segura en el directorio de Git (base de datos local).

2. Implementación y Comandos

Esenciales

Para comenzar un proyecto con Git, primero se debe instalar y configurar la identidad del usuario mediante:

```
git config --global user.name "Tu Nombre"
```

```
git config --global user.email correo@ejemplo.com
```

Tabla de Comandos Frecuentes

Comando	Función
git init	Crea un nuevo repositorio local en el directorio actual.
git clone [url]	Descarga una copia completa de un repositorio remoto.
git status	Informa sobre el estado de los archivos (rastreados, modificados o preparados).
git add [archivo]	Pasa los cambios al área de preparación (Staging Area).

Comando	Función
<code>git commit -m "msj"</code>	Guarda la instantánea de forma permanente en el historial.
<code>git branch [nombre]</code>	Crea una nueva rama de desarrollo aislada.
<code>git checkout [rama]</code>	Cambia de una rama a otra o restaura archivos.
<code>git merge [rama]</code>	Fusiona los cambios de la rama indicada en la rama activa.
<code>git log</code>	Visualiza el historial completo de confirmaciones.

3. Gestión del Progreso: Guardar y Recuperar

¿Cómo guardar el progreso? El guardado es jerárquico:

Localmente: Se usa `git add` seguido de `git commit`.

Remotamente: Se utiliza `git push` para subir las confirmaciones al servidor.

Cambios temporales: Si no queremos confirmar algo aún, `git stash` guarda el estado actual en una pila provisional para recuperarlo luego.

¿Cómo recuperar información?

Podemos hacerlo mediante 4 formas:

Deshacer cambios locales: `git checkout -- [archivo]`
restaura un archivo a su última confirmación
(¡Cuidado! Sobrescribe cambios actuales).

Quitar del área de preparación: `git reset HEAD`
[archivo] desmarca un archivo añadido por error.

Recuperar lo "perdido": `git reflog` permite ver el
historial de movimientos del HEAD para encontrar
SHA-1 de commits que parecen haber desaparecido.

Restaurar el proyecto: `git reset --hard [commit]`
mueve todo el proyecto a un punto específico del
pasado.

4. GitHub: Colaboración y Alojamiento

*GitHub es la plataforma de alojamiento de repositorios
Git más grande del mundo, facilitando la colaboración y
el seguimiento de proyectos.*

Flujo de Trabajo Colaborativo

Fork: Crear una copia personal de un repositorio ajeno
para experimentar libremente.

Pull Request (PR): El mecanismo para proponer tus cambios al dueño del proyecto original. Permite la revisión de código y debate antes de integrar las mejoras.

Gestión y Herramientas

Seguridad: Permite autenticación de dos factores (2FA) y gestión de accesos mediante HTTPS/SSH.

Documentación: Renderiza automáticamente archivos como el **README** (información general) y **CONTRIBUTING** (guías para colaboradores).

Organizaciones: Permite la propiedad compartida de proyectos y la creación de **Equipos** con niveles de permisos específicos.

Automatización: Ofrece **Webhooks** y una **API** potente para integrar servicios externos y automatizar tareas de desarrollo.

Integrantes:

- Francisco Verdi
- Joaquín Chapini
- Valentín Agüero